后端就业学习路线-全面版

《教资展示》

我是谁?

本科绩点最落后之人、但四段大厂实习、秋招某大厂SSP

计算机专业，信息、规划才是最大的鸿沟，跟着学校学习四年大概率只会：**毕业即失业**

随着AI越来越强大，后端开发的工作内容、学习路线也随之变化

通过这个文档分享一些学习路线给在校生，希望对你有帮助

> 🎯 **适用人群**：计算机相关专业（或想转码）的在校学生，目标是拿下后端开发岗
>
> **核心心法**：速通语法 → 尽早进框架 → 项目驱动 → 八股收口
>
> **反面清单**：不在语言细节里溺死；不先背三个月八股；不只收藏不动手

全文共八卷：路线全景与时间预算 → 技术栈（阶段 0~9）→ 计算机基础 → 算法 → 八股 → AI 与 Agent 专题 → 节奏项目与自检 → 附录。**阶段 0~9 是主线的推进顺序**，其余各卷与主线并行推进。

零、Java 、 Golang、Cpp？

岗位通用性：java

大厂、新业务、云原生、转运维：golang+Python一定份额

底层框架、音视频开发：cpp

时间充足、想进阶：java+golang 全套学习、**后端通用八股**

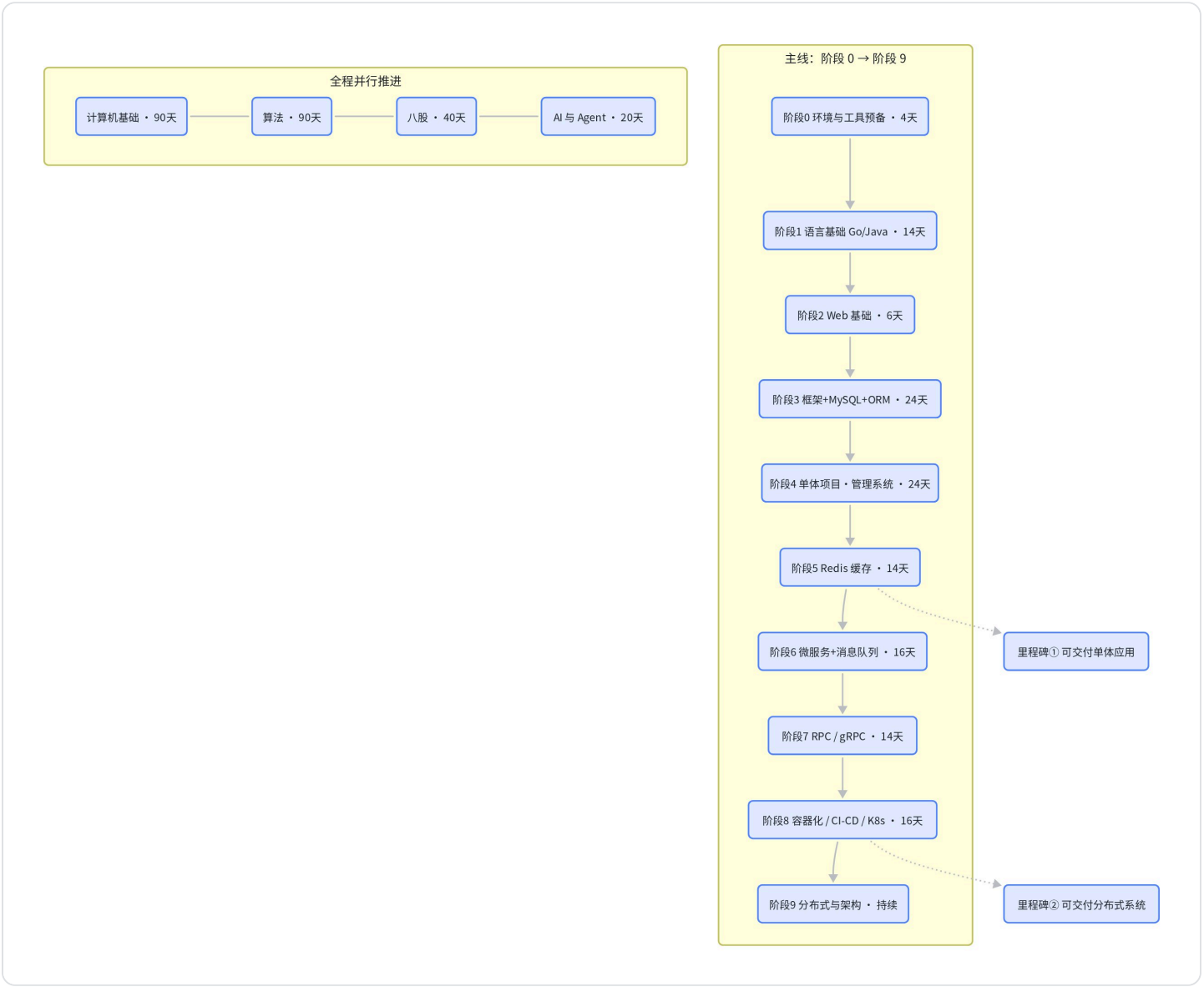
时间紧张：java

tips：

1. 中大厂并不在意候选者语言，更多强调后端通用编码、业务、架构能力（尤其在AI时代）

2. 小厂、央国企、传统行业java偏多，岗位更广泛

一、路线全景与时间预算



两条铁律

1. **项目先行**：每学完一块技术，必须落到一个能跑起来的东西上，否则等于没学。
2. **八股滞后**：八股是「把做过的事讲清楚」的复习材料，不是入门教材。

时间预算（每天 4 小时）

理想情况：每天有效学习 4 小时，按在校学生的作息估算。各行相加等于总计

主线：阶段 0~8

阶段	内容	预计天数	累计天数
0	环境与工具预备	4	4
1	语言基础（Go / Java）	14	18
2	Web 基础	6	24
3	Web 框架 + MySQL + ORM	24	48
4	单体项目（管理系统）	24	72
5	Redis 与缓存思想	14	86
6	微服务思想 + 消息队列	16	102
7	RPC / gRPC	14	116
8	容器化 / CI-CD / K8s	16	132
9	分布式与架构	持续（建议 ⩾ 20）	—
主线小计	**阶段 0~8**	**132**	**132**

并行线 / 专题（与主线同期推进）

模块	内容	预计天数	折算工时
计算机基础	计算机网络 36 + 操作系统 32 + 数据库原理 22	90	360 h
算法	专题 → Hot100 → 高频 200~300	90	360 h
八股	按模块滚动复习，与项目结合	40	160 h
AI 与 Agent	概念辨析 + Agent 辅助开发 + Agent 项目	20	80 h
并行小计	—	**240**	**960 h**

汇总

模块	预计天数	折算工时
主线阶段 0~8	132	528 h
并行线 / 专题	240	960 h
合计	**372 天（≈ 13 个月）**	**1488 h**

💡 **三个关键节点**

- **132 天（≈ 5 个月）**：主线走完，已经能做项目、能上手干活
- **372 天（≈ 13 个月）**：基础 + 算法 + 八股 + AI 全部补齐，是「能过面试 + 会 AI」的完整周期
- 每周按 6~7 天计；寒暑假每天能投 8 小时的，整体可压到 **约 190 天**；选学的 LLM 底层原理再 +6 天

二、技术栈

阶段 0 · 环境与工具预备（4 天）

目标：不因为「环境问题」卡住任何一次学习。

类别	至少要会
命令行	目录切换、文件增删改查、管道、环境变量、进程查看、端口占用排查
Git	init / clone / add / commit / push / pull、分支、merge 与冲突解决、.gitignore、看懂 diff、发一次 PR
编辑器	VS Code + Go 插件，或 IDEA（Java）；会跳转定义、断点调试、格式化
数据格式	JSON 的读写与嵌套、YAML 缩进、Markdown 写作
接口调试	Postman / Apifox / curl，会构造带 header、body、token 的请求
抓包与调试	浏览器 DevTools 的 Network 面板：看请求、响应、耗时、状态码
容器	会用 docker run 起一个 MySQL / Redis（不懂原理没关系）

过关标准

- [ ] 能用 Git 把一个本地项目推到远程仓库，并处理一次冲突
- [ ] 能用 Docker 起 MySQL 并连上建库建表

- [ ] 能读懂浏览器 Network 面板里任意一条请求

阶段 1 · 语言基础：Java + Golang（14 天）

心法：语法速通，**尽快进入框架和项目**。Golang 语法量小，3 小时就能过一遍语法，一两天能写顺手；Java 需要更多时间，但不用太纠结细节 —— 八股特别多，初学者是看不完也看不懂的。

Go 速通清单

分类	必须掌握	可以先跳过
基础	变量与零值、slice / map 的底层行为与扩容、struct 与方法、接口（隐式实现）、error 与 %w 包装、defer / panic / recover	反射、unsafe、汇编
并发	goroutine、channel（有 / 无缓冲）、select、sync.Mutex / WaitGroup / Once、context 传递与取消	sync.Pool 源码、GMP 调度细节
工程	go mod、包组织与命名、gofmt / golangci-lint、表格驱动单测、go test -race 与 benchmark	编译期优化、链接器
标准库	net/http、encoding/json、time、os、io、database/sql	冷门包按需查

学习资源:

1. go-tour:https://go-dev.shuijingwanwq.com/tour/welcome/1
2. go-by-example:https://gobyexample.com/
3. the-book:https://golang-china.github.io/gopl-zh/index.html

Java 速通清单

分类	必须掌握	可以先跳过
语法	类 / 接口 / 抽象类、继承与多态、泛型、枚举、注解、异常体系	内部类骚操作、语法糖细节
集合	List / Map / Set 的选型与区别、HashMap 基本原理、迭代器与并发修改	集合源码逐行精读

函数式	Lambda、Stream、Optional	复杂函数式组合
IO（历史包袱很重）	字节流 / 字符流、BufferedReader、Files、序列化概念	NIO / Netty
反射与注解	会用、知道框架为什么需要它	动态代理实现细节

> 💡 **明确推迟**：JUC 深水区、JVM 内存与调优、类加载机制 —— 这些放到「五、八股」阶段，带着项目经验回头看才看得懂。

学习资源：

1. 黑马程序员java课程
   a. https://www.bilibili.com/video/BV17F411T7Ao
   b. https://www.bilibili.com/video/BV1yW4y1Y7Ms
2. 廖雪峰官方网站（文字版教程）：
   https://liaoxuefeng.com/books/java/introduction/index.html

Go vs Java 心智差异

维度	Go	Java
错误处理	返回值 error，显式判断	异常 try / catch，可中断流程
并发	goroutine + channel，用通信共享内存	线程 + 锁 + 线程池（JUC）
依赖注入	显式传递，无魔法	容器托管（Spring IoC），隐式
面向对象	组合 + 接口，无继承	继承 + 接口 + 注解驱动
构建	go build 单命令	Maven / Gradle 多阶段生命周期

过关标准

- [ ] 用 Go 写一个命令行小工具（如统计目录下文件类型分布）
- [ ] 用 Java 写一个同样的工具，体会两门语言的组织方式差异
- [ ] 能说清「接口」在两门语言中的定位区别

阶段 2 · Web 基础知识（6 天）

目标：搞懂「一个请求从浏览器到数据库再回来」的全过程。HTTP 在这里只需要会用，协议原理不在此深入，留到「三、计算机基础」里精读。

主题	必须掌握
HTTP	请求 / 响应结构、常用方法与幂等性、状态码语义、常见头部、Content-Type
缓存	Cache-Control、ETag / If-None-Match、强缓存与协商缓存
连接	TCP 三次握手 / 四次挥手、HTTPS 握手大致流程、HTTP/1.1 keep-alive、HTTP/2 多路复用
跨域	同源策略、CORS 简单请求与预检请求
会话	Cookie / Session / Token / JWT 的区别与取舍
数据交换	JSON 序列化、编码问题、时间格式约定
HTTP 服务器	静态资源、反向代理、Nginx 的作用与常见配置（了解即可）
架构思想	前后端分离：后端只出接口，前端负责渲染，接口契约先行
设计规范	RESTful 资源命名、分页 / 排序 / 过滤约定、统一错误码
Java 套件额外补充	Servlet 生命周期、Tomcat 是什么（容器 + 连接器 + 线程模型），了解即可

会话方案对比

方案	状态存放	优点	缺点与适用
Cookie	客户端	简单	可篡改、体积限制；适合轻量标记
Session	服务端	可控、可注销	多机需共享（Redis）；适合传统单体
Token	客户端	无状态、跨服务	无法主动失效；适合 API 服务
JWT	客户端（自包含）	免查库、可携带声明	无法撤销、需管理密钥与过期；适合微服务鉴权

过关标准

- [ ] 能画出一个登录请求的完整链路（含 Cookie 下发与携带）
- [ ] 能用 curl 手动构造一个带 token 的请求并解释每一行
- [ ] 能说清「预检请求什么时候触发」

阶段 3 · 一门 Web 框架 + MySQL + ORM（24 天）

✅ **强烈建议先用 golang-gin，而不是一上手就 SpringBoot。**通过 Gin 学会项目管理、依赖管理、HTTP 请求响应、路由、日志、ORM 等后端开发的常见知识。

Gin 学习清单

主题	内容
路由	路径参数、查询参数、Group 分组、中间件链
请求处理	参数绑定与校验（ShouldBind + validator tag）、文件上传
响应	统一响应结构、错误码设计、全局异常 / panic 恢复
中间件	日志、鉴权、跨域、限流、context 传值
工程化	分层（handler / service / repository）、配置文件（yaml + env）、依赖注入（显式）
日志	结构化日志（zap / slog）、请求 trace id、分级输出
数据库	database/sql 连接池参数、GORM 基本用法
ORM 常见坑	N+1 查询、零值更新、软删除、事务传播、批量插入
测试	httptest、表格驱动单测、测试库隔离

MySQL 学习清单

层次	必须掌握
会用	建库建表、CRUD、JOIN、聚合分组、子查询、分页
表设计	字段类型选择、范式与反范式、索引设计、唯一约束、逻辑删除
索引	B+ 树结构直觉、最左前缀、覆盖索引、回表、索引失效场景
事务	ACID、四种隔离级别、脏读 / 不可重复读 / 幻读、MVCC 直觉
性能	EXPLAIN 读懂 type / key / rows / Extra、慢查询日志、SHOW PROFILE

运维	备份与恢复、主从复制概念、分库分表先了解不深入

从 Gin 铺垫到 SpringBoot（java）

先在 Gin 里体会「依赖是手动传进去的」（**显式**），再回头看 Spring 的**控制反转、依赖注入**就非常自然。

主题	内容
构建	Maven 坐标（groupId / artifactId / version）、依赖传递、生命周期（clean / compile / test / package）
核心(反射-注解)	@SpringBootApplication、自动配置原理（条件注解 + starter）、application.yml
IoC / DI	Bean 的生命周期与作用域、@Component / @Service / @Repository、构造器注入优先
AOP	切面、通知类型、事务 @Transactional 的失效场景
持久层	MyBatis 半自动化 ORM：Mapper 接口、XML / 注解、动态 SQL、#{} 与 ${} 区别
Web 层	@RestController、参数绑定、全局异常处理、拦截器 / 过滤器
测试	@SpringBootTest、MockMvc

过关标准

- ☐ 用 Gin + MySQL 写一套完整的用户 CRUD 接口（含校验、分页、统一响应）
- ☐ 用 SpringBoot + MyBatis 复刻同一套接口
- ☐ 能口述「Gin 的中间件链」和「Spring 的 AOP」分别解决了什么问题

阶段 4 · 构建一个有一定复杂度的单体项目：管理系统（24 天）

管理系统是初学者绕不开的，它确实包含了很多知识。

功能清单（按优先级）

优先级	模块
P0	登录 / 登出、JWT 鉴权、用户管理 CRUD、分页搜索

P0	角色与权限（RBAC：用户-角色-权限）、接口级鉴权中间件
P1	菜单 / 资源管理、操作审计日志、数据字典
P1	文件上传（本地 / 对象存储）、Excel 导入导出
P2	通知公告、定时任务、数据统计看板

技术清单

- 分层架构：handler → service → repository → model
- 参数校验、统一响应体、全局异常处理、错误码表
- 日志（结构化 + trace id）、配置管理（多环境）、数据库迁移脚本
- 缓存（可选，用于字典 / 权限等热点数据）
- 接口文档：Swagger / OpenAPI
- 单元测试 + 关键流程集成测试
- docker-compose 一键起服务（app + mysql + redis）

软技能（比技术更容易被忽略）

能力	具体要求
需求文档	能把一句话需求拆成功能点、边界条件、异常分支
API 文档	写得出别人不用问就能对接的接口文档
Git 工作流	分支命名规范、小步提交、写清楚 commit message、会发 PR
Code Review	能读懂别人的 diff，也能接受批评
交付意识	README 写得清楚：怎么跑、怎么测、有哪些坑

AI 时代的用法

好处很实在：可以让 AI 直接快速生成一个可用的前端，方便测试和成果展示。

- 让 AI 生成前端页面、样板代码、单测、SQL、文档草稿
- 让 AI 做 code review，指出边界问题和安全漏洞
- **红线**：核心业务逻辑、数据流、鉴权必须自己写并讲得清楚 —— 面试只考你脑子里的东西

完整方法论（Spec 先行 → 架构设计 → TDD、各环节怎么用、红线清单）见「六、AI 与 Agent 专题」。

过关标准

- [ ] 项目能 docker-compose up 一键运行，README 里有完整截图
- [ ] 有 ≥ 15 个接口、≥ 5 张表、完整的鉴权链路
- [ ] 能对着 API 文档给同学讲 20 分钟，对方能听懂
- [ ] 有 ≥ 10 个单测，且 go test -race 通过

阶段 5 · Redis（缓存）（14 天）

目标：建立「缓存思维」。此时还不太需要深入到 Redis 原理，常用数据结构在业务中的应用才是重点。

数据结构 → 业务场景映射

结构	典型场景
String	计数器、验证码、分布式锁、对象缓存
Hash	用户信息、购物车
List	简易消息队列、最新列表、时间线
Set	点赞去重、共同关注、抽奖池
ZSet	排行榜、延时队列、热度排序
扩展	GEO（附近的人）、HyperLogLog（UV 统计）、Bitmap（签到、活跃统计）

核心问题清单

问题	解决思路
缓存穿透	空值缓存、布隆过滤器、参数校验
缓存击穿	互斥锁重建、逻辑过期
缓存雪崩	过期时间加随机、多级缓存、熔断降级
双写一致	Cache Aside + 延迟双删、订阅 binlog 更新缓存
~~分布式锁~~	~~SET NX PX → 看门狗续期 → Redisson；注意锁误删与可重入~~
内存管理	过期删除策略、8 种淘汰策略、大 key / 热 key 治理
持久化	RDB 与 AOF 的取舍（先懂结论，原理放八股）

推荐项目

黑马点评：虽然有点旧了，但后续可以基于此重新写一个类似的项目，主要为了学习**缓存的思想**。用 Gin 重做一遍时，可加入以下加分项：

- 秒杀下单：Redis + Lua 原子扣减 + 消息队列异步落库
- 点赞 / 关注 / 共同关注：Set 与 ZSet
- 附近商铺：GEO；用户 UV：HyperLogLog
- 缓存三兄弟的完整防护 + 压测数据对比（缓存前后 QPS、P99）

里程碑 ① · 已经可以构建不错的单体应用了

- [ ] 能独立从 0 到 1 交付一个带鉴权、缓存、文档、部署的单体系统
- [ ] 能解释系统里每一条数据的来源与流向
- [ ] 能为它做一次压测，并说清瓶颈在哪

阶段 6 · 微服务思想 + 消息队列（16 天）

第一步：先有微服务的思想

1. **根据职责拆分服务**：按业务域（用户、订单、商品、支付）而不是按技术层。
2. **普通接口先用 HTTP 互相调用** —— 此时还没学 RPC，先把「拆分」这件事跑通。
3. 体会拆分带来的问题：调用链路变长、事务跨服务、数据一致性、排障困难。

第二步：消息队列

学习消息队列的思想，**解耦服务调用**（削峰填谷是额外的收益）。

选型	吞吐	延迟	顺序 / 事务	学习建议
Kafka	极高	低	分区内有序 / 支持	想学「流」和日志型场景
RabbitMQ	中	极低	队列内有序 / 需变通	**入门首选**，概念最清晰
RocketMQ	高	低	支持 / 原生支持	业务型场景，支付 / 订单

选一个学透即可：概念是通用的，API 是次要的。必会概念：生产者 / 消费者 / Topic / 分区 / 消费组 / offset / ack / 重平衡。

必须想清楚的问题	手段
消息重复	消费端幂等（唯一键 + 去重表 / 状态机）

消息丢失	生产确认、持久化、手动 ack
消息顺序	同 key 路由同分区、单消费者串行
消息积压	扩容消费者、提高并发、限流保护
死信处理	重试次数上限 + 死信队列 + 人工兜底

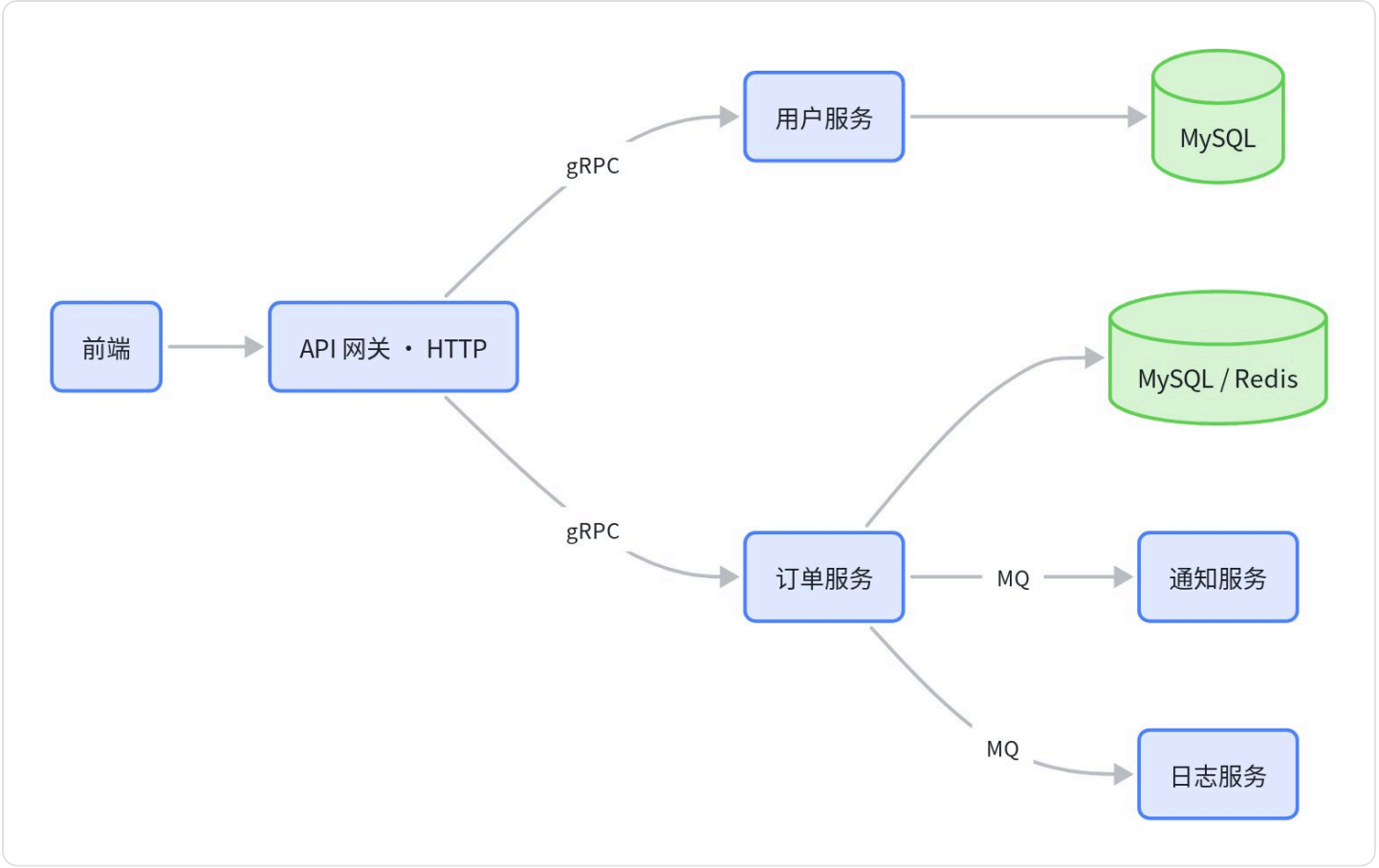
实践目标：把阶段 4 的系统里的「下单 / 通知 / 日志」改成异步消息驱动。

阶段 7 · RPC 与 gRPC（14 天）

有了微服务思想后，学会 gRPC、学会 protobuf，用 gRPC 的方式让多个服务通讯，而不是裸 HTTP。

主题	内容
为什么用 RPC	二进制编码体积小、IDL 契约先行、代码自动生成、原生流式、强类型
protobuf	.proto 语法、message / service、字段编号与向后兼容、optional / repeated / oneof
四种模式	一元、服务端流、客户端流、双向流
工程化	拦截器（日志 / 鉴权 / 重试 / 埋点）、超时与 deadline、metadata 传参、错误码映射
服务治理	服务注册与发现（etcd / Consul）、客户端负载均衡、健康检查、优雅关闭
与 HTTP 共存	grpc-gateway 暴露 REST 接口给前端

综合项目：把单体系统拆成 3~4 个服务，服务间用 gRPC，异步用 MQ，热点数据用 Redis，对外用 HTTP 网关。



过关标准

- [ ] 至少两个服务通过 proto 定义契约并互相调用
- [ ] 有统一的拦截器处理日志、鉴权、超时
- [ ] 能解释「为什么这里用 gRPC 而不是 HTTP」

阶段 8 · 容器化、CI/CD、K8s（16 天）

主题	必须掌握
Docker	镜像分层、Dockerfile 指令、多阶段构建、.dockerignore、数据卷、自定义网络、docker-compose
CI/CD	流水线思路：lint → test → build → 镜像推送 → 部署；GitHub Actions / GitLab CI 任选其一
K8s 核心对象	Pod、Deployment、Service、Ingress、ConfigMap、Secret、PVC、Job / CronJob、HPA
K8s 操作	kubectl 常用命令、滚动更新与回滚、探针（liveness / readiness）、资源 requests / limits
打包部署	Helm Chart 基础、命名空间隔离、镜像 tag 策略

可观测	指标 Prometheus + Grafana、日志 Loki / ELK、链路 Trace（OpenTelemetry）

过关标准

- [ ] 每个服务都有多阶段构建的镜像，体积控制在合理范围
- [ ] 一次 push 能自动跑测试、构建镜像并部署到测试环境
- [ ] 在本地 K8s（kind / minikube）里跑起整套系统，能看板看到 QPS 与错误率

阶段 9 · 分布式与架构（进阶 · 持续）

分布式理论

主题	要点
CAP / BASE	三者不可兼得、最终一致性的工程取舍
一致性协议	Raft（能讲清选举与日志复制）、Paxos / ZAB（了解）
分布式事务	2PC / 3PC、TCC、Saga、本地消息表、最大努力通知、Seata 用法与代价
分布式 ID	雪花算法、号段模式、时钟回拨问题
分布式锁	Redis 实现与 Redlock 争议、ZooKeeper / etcd 实现
稳定性	限流、熔断、降级、隔离、重试退避、幂等、超时预算

架构能力

- 演进路径：单体 → 模块化单体 → 微服务 → 事件驱动 / 云原生
- **判断力**：什么时候不该上微服务（团队规模、运维能力、一致性成本）
- DDD 战术设计：实体、值对象、聚合、仓储、领域事件、限界上下文
- 高可用设计：无状态化、冗余、故障域隔离、容量评估与压测（wrk / vegeta / k6）
- 性能优化方法论：先测量再优化 —— 指标（QPS / RT / P99 / 资源水位）→ 定位 → 假设 → 验证

可选纵深方向

按兴趣与岗位选一个：Elasticsearch 搜索、对象存储与 CDN、WebSocket / IM、任务调度平台、Service Mesh、Serverless、网关二次开发、数据平台与 AI 应用结合。

三、计算机基础

这三门是「决定天花板」的部分，贯穿全程，效果好于一学期突击。

计算机网络

1. **通读**《计算机网络：自顶向下方法》—— 建立整体图景，不纠结细节
2. **精读 应用层、TCP、IP 三层**：HTTP 熟练应用，TCP、IP 八股背熟

层次	精读要点
应用层	HTTP 报文、方法与状态码、Cookie、缓存、DNS 解析、HTTPS / TLS 握手、HTTP/2 与 HTTP/3
传输层	TCP 三次握手 / 四次挥手、可靠传输、滑动窗口、拥塞控制、TIME_WAIT、UDP 与 QUIC
网络层	IP 编址与子网、路由与 NAT、ICMP、ARP
工具	Wireshark / tcpdump 抓一次真实的三次握手与 TLS 握手

实验建议：用 Wireshark 抓一次访问 HTTPS 网站的全过程，能对着包讲出每一步在干什么。

学习资源：

1. https://www.bilibili.com/video/BV1JV411t7ow
2. https://cs144.github.io/

操作系统

1. **通读教材**，如《Operating Systems: Three Easy Pieces》（OSTEP）
2. **精读并熟练掌握**：

模块	精读要点
进程与线程	进程模型、上下文切换、线程与协程、进程间通信
调度	常见调度算法、优先级与时间片、nice
内存	虚拟内存、分页与缺页中断、页面置换、栈与堆、内存分配
并发	互斥与同步、锁、信号量、条件变量、死锁四条件与排查

文件系统	inode、目录结构、读写路径、缓冲与页缓存、磁盘调度
IO 模型	阻塞 / 非阻塞 / IO 多路复用（select、poll、epoll）/ 异步 IO

实验建议：写一个简易 shell、实现一个线程池、手写一个 LRU 缓存、用 strace 看一次系统调用序列。

数据库原理

在学习 MySQL 使用之上补齐原理：索引为什么是 B+ 树、事务如何用 undo / redo / binlog 实现、MVCC 与 Read View、锁的类型与加锁规则、日志两阶段提交。

学习资源：

1. https://www.bilibili.com/video/BV1Kr4y1i7ru
2. https://cs145.web.app/

数据结构与基础算法

刷题，狠狠刷，这些都是基础中的基础。

- 关注up **灵茶山艾府**：https://space.bilibili.com/206214 （以及其在leetcode的账号）
- 先把六大件过一遍：数组与字符串、链表、栈与队列、哈希表、树、图
- 再掌握八大工具：排序、二分、双指针 / 滑动窗口、递归与回溯、DFS / BFS、动态规划、贪心、并查集
- 要求：**能手写**，不是「看懂了」

其他专业课程：计算机组成原理、编译原理（离散数学）（了解即可，不必深究）。

四、算法

预计 90 天。此部分必须先完成前面说的数据结构与基础算法。建议节奏：每天 1 小时（约 2~3 题），与主线并行，从阶段 1 结束就开始，不要攒到最后。

步骤	目标	题量参考	预计天数
① 语法熟练期	用目标语言顺手写出常见结构，不再卡语法	30~50 题	10
② 专题突破期	每个专题吃透模板与变体，能讲出思路	100~150 题	30
③ Hot100	**hot100 狠狠刷**，覆盖面试高频模型	100 题，二刷三刷	24
④ 高频冲刺	刷完 hot100，**高频题再刷个两三百道就行了**	200~300 题	26

专题清单

1. 数组 / 字符串：双指针、滑动窗口、前缀和、差分
2. 哈希表与集合去重
3. 二分查找（含边界与答案二分）
4. 链表：反转、快慢指针、合并、环
5. 栈与队列：单调栈、单调队列、括号匹配
6. 树：遍历、BST、LCA、路径问题
7. 回溯：排列 / 组合 / 子集 / 剪枝
8. 动态规划：线性、区间、背包、状态机、树形
9. 图论：DFS / BFS、拓扑排序、最短路（了解）
10. 并查集、位运算、堆与 TopK

方法（比题量更重要）

- 限时：简单 10 分钟、中等 25 分钟、困难 40 分钟，超时看题解
- 看题解后**必须盲写一遍**，隔 3 天、7 天各复习一次
- 建错题本：记「思路卡点」而不是记代码
- 面试手撕注意：先讲思路 → 再写代码 → 最后主动测边界（空、单元素、重复、溢出）

五、八股

八股的本质是「把你做过的事，用准确的术语讲清楚」，学完对应技术后立刻复习效果最好。建议节奏：前期每天 0.5~1 小时滚动复习（学完一块技术就复一块），最后 40 天集中冲刺与模拟面试。

模块	高频考点	目标
MySQL + Redis	索引与优化、事务与锁、MVCC、主从与分库分表；Redis 数据结构、持久化、缓存三兄弟、分布式锁	能结合项目举例
JUC	线程池参数与拒绝策略、synchronized 与 AQS、volatile、CAS、并发容器	能说清取舍
JVM	内存区域、对象创建、GC 算法与收集器、类加载、常见调优与 OOM 排查	能画图讲解
计算机网络	TCP 状态机、握手挥手、拥塞控制、HTTP 版本差异、HTTPS	背熟 + 能画时序
操作系统	进程线程、调度、内存管理、锁与死锁、IO 多路复用	能联系到 Go / Java
框架与工程	Gin / Spring 原理、IoC 与 AOP、MyBatis、Git、Linux 常用命令	能讲自己的理解
分布式与架构	CAP、分布式事务、限流熔断、消息队列可靠性、服务治理	结合项目讲

八股可参考 小林coding：https://xiaolincoding.com/

复习方法

- 费曼法：对着空气讲一遍，卡住的地方就是没懂的地方
- 画图法：把 TCP 握手、MVCC、GC 流程画成图，比背文字牢固十倍
- 项目锚定：每个八股点都找一个自己项目里的例子，面试时讲成「我怎么解决的」
- 拒绝纯背诵：面试官追问两层就能识破背诵痕迹

六、AI 与 Agent 专题

预计 20 天（+ 6 天选学）· **融入在日常学习中**，不要攒到最后突击：概念随用随查（2 天）、Agent 辅助开发从阶段 1 就开始天天用（4 天）、Agent 项目放在阶段 7 之后做（14 天）。

为什么后端要学这个

- 现在后端岗位普遍要求「会用 AI 工具」+「能把 LLM 接进业务系统」，这是简历上最新的差异化亮点

- AI 应用的后端 = 传统后端能力（并发、缓存、稳定性、工程化）+ LLM 编排，**你的后端功底直接决定 AI 项目的质量**
- 最容易做出「有点意思的个人项目」的方向，也是最好和已有项目合并讲述的方向

学习顺序建议

时间点	做什么	对应小节
阶段 1 开始	把 Agent 当成日常工具：生成样板、写单测、解释报错	用 Agent 辅助开发
阶段 2~3	顺手补概念：Prompt、Token、上下文窗口、幻觉	名词辨析
阶段 4	用它生成前端和文档，做 Code Review	用 Agent 辅助开发
阶段 5~7	补 RAG、Embedding、向量库、Function Calling、MCP	名词辨析 + Agent 项目
阶段 7 之后	集中 14 天做一个 Agent 项目	Agent 项目
有空再学	LLM 底层原理（Transformer 入门）	选学小节

AI、LLM、Agent 名词辨析与概念掌握（2 天）

目标：**能用一句话说清每个词，并画得出它们之间的关系**。面试被问到不卡壳，做项目时知道该查哪个。

模型与基础概念

名词	一句话解释	你需要知道的
AI / ML / DL	逐层收窄的三个领域：人工智能 ⊃ 机器学习 ⊃ 深度学习	只需知道层级关系
基模（Foundation Model）	大规模预训练出来的通用模型，可被下游复用	GPT / Claude / DeepSeek / Qwen / Llama 都是
LLM	大语言模型，本质是「根据上下文预测下一个 token」	由此推出：为什么有幻觉、为什么上下文质量决定输出质量
Token / 上下文窗口	模型处理文本的最小单位 / 一次能装多少 token	关系到计费、截断策略、长文本怎么处理
Prompt	给模型的输入指令	

		分清系统提示（设定角色与规则）与用户输入
温度 / Top-p	控制采样随机性的参数	写代码、做抽取要低温；写文案可以高温
幻觉	模型一本正经地编造事实	靠 RAG、工具调用、结果校验缓解，不能只靠提示
Embedding	把文本映射成向量，语义相近则向量相近	RAG 与语义检索的基础
向量数据库	存向量并做相似度检索的库	pgvector / Qdrant / Milvus / Chroma
RAG	检索增强生成：先查资料，再让模型基于资料回答	解决「模型不知道你公司内部的事」
Function Calling / Tool Use	模型按约定格式要求调用你写好的函数	Agent 的「手脚」，也是把 LLM 接进后端的标准做法
MCP	Model Context Protocol，统一「Agent 怎么连接工具与数据」	接外部系统的通用接口，重点掌握
A2A	Agent 与 Agent 之间的协作协议	多 Agent 协作的通信层，了解即可

Agent 相关概念（重点）

名词	一句话解释	你需要知道的
Agent	LLM + 工具 + 记忆 + 循环决策，能自己拆任务、调工具、看结果再继续	和「一次问答」的区别在于**有循环和工具**
Harness	把模型、工具、上下文、权限组织起来的运行时外壳	你在用的这类编程 Agent 工具就是一个 harness
Skill	可复用的能力封装，按需加载给 Agent	不要把所有指令一次性塞进上下文，按需加载才省 token、不跑偏
ReAct	思考（Reason）→ 行动（Act）→ 观察（Observe）的循环范式	自己实现一个最小 Agent 就要用到它
工作流 vs Agent	工作流是你写死的流程；Agent 是模型自己决定下一步	能用工作流稳定解决的，不要上 Agent
多 Agent	多个 Agent 分工协作（规划者 / 执行者 / 评审者）	先会单体 Agent，再考虑多 Agent，否则调试会失控

过关标准

- [ ] 能一句话说清：基模 / Harness / Agent / Skill / MCP / A2A 各自是什么、如何套在一起
- [ ] 能画出「用户 → Agent（循环）→ 工具 / 检索 → 数据源」的流程图
- [ ] 能说清 RAG 与微调各自解决什么问题、什么时候选哪个（多数业务场景 RAG 就够）
- [ ] 能说清「工作流」和「Agent」的边界

如何使用 Agent 辅助开发（4 天，之后每天都要用）

核心方法论：**Spec 先行 → 架构设计 → TDD**。其中 TDD 尤其值得用 AI 做，因为**让 AI 编写单测比让它写实现更可靠**，而单测又是防止它写偏的护栏。

工作流

步骤	具体做法	为什么
① Spec 先行	先写清输入 / 输出、边界条件、错误码、验收标准，再让它写代码	Spec 越具体，产出越可用；Spec 本身就是最好的提示词
② 架构设计	让它给出 2~3 个方案与取舍，**由你拍板**	架构决策不能外包
③ 先测后码（TDD）	让它先按 Spec 写单测（红）→ 再写实现（绿）→ 一起重构	AI 写单测比写实现更可靠，且能自动验证实现
④ 小步验证	每完成一个小任务就跑测试、跑构建	早发现问题，避免一次性生成几百行看不懂的代码
⑤ 提交留痕	小步提交，写清 commit message	随时可回滚，出问题能定界

各环节怎么用

环节	怎么用	注意
需求拆解	把一句话需求拆成功能点、边界、异常分支	你负责确认有没有漏
样板代码	CRUD、DTO、配置、脚手架、SQL、迁移脚本	省时间的主要来源
单测与用例	表格驱动单测、边界用例、Mock 数据	**首选用法**，收益最高
读陌生代码	让它解释项目结构、画调用链、找入口	结论自己再验证一遍
Debug	贴报错 + 最小复现，让它提假设，逐个验证	别让它「猜着改」
Code Review	检查边界、并发、SQL 注入、密钥泄露	你仍是最终责任人

文档	README、API 文档、注释、提交信息	生成后自己校对
前端与演示	快速生成一个能用的界面，方便测试与成果展示	重点仍是后端逻辑

用法要点

- **上下文给足**：相关文件、报错、约束、技术栈一次说清，比反复纠正快得多
- **任务拆小**：一个任务一个目标；大任务先让它出计划，你确认后再执行
- **要求自验证**：让它自己跑测试 / 构建并贴出结果，而不是声称「写好了」
- **善用规则文件**：把项目规范（分层、命名、错误码、禁用写法）写成规则文件，让它每次都遵守
- **保持可回滚**：改动前先提交，随时能 git checkout

> ⛔ **红线（踩了会出事）**
>
> - 提交自己看不懂、讲不清的代码 —— 面试两轮追问就会露馅
> - 把密钥、真实用户数据、公司内部资料贴给外部模型
> - 让它替你决定架构、关键算法、数据模型
> - 不跑测试就合并；盲目接受它引入的新依赖（供应链投毒风险）
> - 用 AI 生成的内容冒充自己的思考与成果

过关标准：能独立用 Agent 完成「写 Spec → 出单测 → 写实现 → 自测通过 → 提交」一整轮，并说清每一步你做了什么判断。

Agent 项目（14 天）

学习主流 Agent 框架与思想，如 **LangChain**、**Eino**，挑选一个编写一个 Agent 项目。

技术栈路径

层次	内容
模型接入	OpenAI 兼容协议、流式输出（SSE）、多模型切换、超时与重试、成本统计
提示工程	系统提示设计、Few-shot、结构化输出（JSON Schema / 强制工具调用）
工具调用	Function Calling、参数校验、工具白名单与权限、失败重试与兜底
记忆	短期（会话历史 + 截断 / 摘要）、长期（向量库检索）
RAG	文档解析 → 切分 → Embedding → 向量库（pgvector / Qdrant / Milvus）→ 召回 + 重排

Agent 循环	ReAct（思考 → 行动 → 观察 → 再思考）、计划-执行、评审回路
协议	MCP（接工具与数据）、A2A（Agent 间协作，了解）
后端工程化	并发与限流、配额计费、缓存、幂等、异步任务、优雅降级
可观测	trace：每次调用的 token、耗时、成本、工具调用链
评测（Eval）	20~100 条评测集，改动前后对比效果，避免「感觉变好了」
安全	提示注入、越权工具调用、输出过滤、数据脱敏

选题（挑一个，做出能演示的版本）

选题	用到的能力	后端亮点
企业知识库问答	RAG + 引用溯源	文档流水线、向量检索、权限隔离
智能 SQL 助手	Function Calling + 表结构理解	只读校验、SQL 审计、结果解释
代码审查机器人	长上下文 + 结构化输出 + Webhook	异步任务、Git 集成、评论回写
智能客服 / 工单分类	意图识别 + 工具调用 + 会话状态	多轮状态机、人工兜底、数据看板
给阶段 4 的管理系统加一个「AI 助手」入口	工具调用 + RAG	最省力，且能和已有项目合并讲述

过关标准

- [ ] 有完整的链路（RAG 或工具调用），不是只有一个聊天框
- [ ] 处理了超时、重试、限流、成本统计中的至少三项
- [ ] 有评测集，能说明改动前后的效果变化
- [ ] 能讲清「为什么这里用 Agent / RAG，而不是一次简单的模型调用」
- [ ] 能说清这套系统的瓶颈在哪（延迟、成本、还是召回质量）

（选学）了解 LLM 底层原理：入门 Transformer（6 天）

目的是**看懂名词、面试能接话**，不是转行做算法。有兴趣再学，没兴趣可以跳过。

主题	学到哪为止
Tokenization	BPE 分词的直觉：为什么中文更费 token

注意力机制	Self-Attention 在算什么、为什么需要位置编码
Transformer 结构	Encoder / Decoder、多头注意力、残差与 LayerNorm、FFN
训练三阶段	预训练 → SFT → RLHF / DPO
推理与成本	KV Cache、量化、批处理，显存与吞吐的关系
微调	LoRA / 全参微调、数据准备，什么场景才值得做

可选加分：用 Go 或 Python 手写一个最小的注意力计算；读一遍某个开源 Agent 框架的核心循环源码。

这一卷的产出物

产出	说明
一份自己的「AI 名词卡片」	30 个词，每个一句话 + 一个项目中的例子
一套 Agent 协作规范	写进你的项目：Spec 模板、单测要求、禁用写法、提交规范
一个能演示的 Agent 项目	带评测集与成本 / 延迟数据，放进简历与作品集

七、节奏、项目与自检

每天 4 小时怎么分配

日常模板（周一到周六，每天 4 h）

顺序	内容	时长	说明
①	算法刷题	1 h	固定开场，脑子最清醒时刷 2~3 题
②	主线技术学习 + 动手写代码	1.5 h	当天阶段的核心任务，必须写出可运行的东西
③	计算机基础 / 八股 / AI	1.5 h	拆成约 1 h 基础 + 25 min 八股 + 10 min AI 卡片，可碎片化

每周配比（24 h = 6 天 × 4 h，与时间预算表对齐）

模块	每周	折合每天	对应卷
主线技术 + 项目	8.5 h	1.5 h	二

计算机基础	6 h	1 h	三
算法刷题	6 h	1 h	四
八股	2.5 h	25 min	五
AI 与 Agent	1 h	10 min	六
合计	**24 h**	**4 h**	—

项目周 / 冲刺周可以临时倾斜：把基础与八股压到 0，集中火力推进阶段 4 或 Agent 项目，但**算法不要断**。

每周留 1 天（周日）：集中推进项目（4 h 不被打断的整块时间，用于啃硬骨头）；复盘本周学到了什么、卡在哪、下周做什么。

给在校生的四条时间建议

- 没课的整块时间用来写代码和做项目，碎片时间（课间、排队）用来看概念和刷选择题
- 期末周 / 课程设计周允许主线停摆，但**算法不断线**（每天 1 题也行）
- **AI 工具要天天用**：从阶段 1 写第一个 CRUD 就开始，用满 372 天，比最后花 20 天速成有用得多
- 每完成一个阶段就打勾过关清单，比按日历推进度更靠谱

项目作品集建议

项目	体现的能力
单体管理系统（阶段 4）	工程规范、CRUD、鉴权、文档能力
缓存实战项目（阶段 5）	高并发思维、Redis 应用、压测与优化
微服务系统（阶段 7）	服务拆分、RPC、MQ、分布式问题处理
Agent 应用（六）	LLM 编排、RAG、工程化与评测，最新的差异化亮点
一个「有点意思」的个人项目	热情与自驱（爬虫 / 工具 / 小游戏后端都可以）

简历写法：用「背景 → 问题 → 方案 → 数据」四段式，量化结果（QPS 从 300 提升到 2000、P99 从 800ms 降到 120ms；Agent 项目可量化召回准确率、平均延迟与单次成本）。

常见坑

- ❌ 收藏了几十 G 的教程，一行代码没写
- ❌ 语言语法学了三个月还没碰框架

- ❌ 一开始就背 JVM / JUC 八股，看不懂也记不住
- ❌ 项目只看视频跟敲，不会自己加需求、不会排错
- ❌ 只写代码不写文档，面试讲不清楚
- ❌ 一上手就上微服务 / K8s，基础没打牢
- ❌ 把 AI 当答案生成器：代码能跑就算完，自己讲不出为什么
- ❌ 为了「蹭 AI」去学算法岗的内容（Transformer 推导、手撕反向传播），偏离后端主线

总自检清单

语言与 Web

- [ ] 能用 Go 与 Java 各写一个可运行的服务
- [ ] 能解释一次 HTTP 请求的完整生命周期
- [ ] 能说清 Cookie / Session / Token / JWT 的取舍

数据库与缓存

- [ ] 能设计一张表并说明索引选择理由
- [ ] 能用 EXPLAIN 定位慢查询
- [ ] 能实现缓存三兄弟的完整防护
- [ ] 能用 Redis 实现分布式锁并说明风险

工程与交付

- [ ] 项目可一键启动，有 README 与 API 文档
- [ ] 有 CI 流水线，代码有测试
- [ ] 能把服务容器化并部署到 K8s

分布式

- [ ] 能画出一个微服务系统的架构图并讲清数据流
- [ ] 能说明消息重复 / 丢失 / 顺序的处理方案
- [ ] 能对比至少两种分布式事务方案

基础与算法

- [ ] 网络 / OS 核心问题能画图讲解
- [ ] Hot100 能独立写出 80% 以上
- [ ] 八股能结合项目讲，而不是背定义

AI 与 Agent

- [ ] 能用一句话说清基模 / Harness / Agent / Skill / MCP / A2A 的关系

- [ ] 能用 Agent 完成「Spec → 单测 → 实现 → 自测 → 提交」一整轮
- [ ] 有一个能演示的 Agent 项目，含评测集与延迟 / 成本数据
- [ ] 能说清什么时候该用 RAG、什么时候该用工作流而不是 Agent

八、附录

资源清单

类别	资源
计算机网络	《计算机网络：自顶向下方法》
操作系统	《Operating Systems: Three Easy Pieces》(OSTEP)
算法教程	灵茶山艾府 https://space.bilibili.com/206214
八股	小林coding https://xiaolincoding.com/
项目实战	黑马点评（缓存思想）
AI / Agent	主流 Agent 框架官方文档（LangChain、Eino）、MCP 规范文档、各家模型 API 文档
官方文档	Go 官方 Tour、Gin、GORM、Spring Boot、gRPC、Docker、Kubernetes
刷题平台	LeetCode（Hot100 / 面试经典 150）

术语速查

术语	含义
IoC / DI	控制反转 / 依赖注入：对象由容器创建并注入，而非自己 new
ORM	对象关系映射：用对象操作数据库表
RBAC	基于角色的访问控制
CAP	一致性、可用性、分区容错性三者不可兼得
MVCC	多版本并发控制，读不加锁的关键
MQ	消息队列
RPC	远程过程调用

IDL	接口描述语言（如 protobuf）
CI/CD	持续集成 / 持续交付部署
QPS / RT / P99	每秒查询数 / 响应时间 / 99% 请求的响应时间上限
LLM	大语言模型，本质是预测下一个 token
Agent	LLM + 工具 + 记忆 + 循环决策，能自主拆解任务并调用工具
Harness	把模型、工具、上下文与权限组织起来的运行时外壳
Skill	按需加载给 Agent 的可复用能力封装
MCP	模型上下文协议，统一 Agent 连接工具与数据的接口
A2A	Agent 之间的协作协议
RAG	检索增强生成：先检索资料，再让模型基于资料回答
Embedding / 向量库	文本的向量表示 / 存向量并做相似检索的数据库

> 这份路线图的价值不在「读完」，而在「照着做」。每天推进一小步，比收藏一百份路线图有用得多。